Rapport technique Apprentissage par Renforcement

Simulation de Course Automobile

SAE S5.A.01 - BUT Informatique
Tuteur technique : M. DESBENOIT Brett
Volume horaire : 195 heures

Table des matières

1. Introduction....3
2. Étude Préliminaire et Choix Technologiques.... 4
2.1 Le moteur physique.... 4
2.2 Le module d'intelligence Artificielle....4
2.3 Le pont entre l'IA et le moteur physique.... 5
2.4 Environnement de développement et automatisation.... 6
3. Architecture.... 7
3.1 Arborescence.... 7
3.2 Flux de Données et Pipeline d'Entraînement.... 8
4. Implémentation....10
4.1 Physique....10
4.1.1 Collisions.... 10
4.1.2 Adhérence Latérale.... 10
4.1.3 Simulation de la direction....11
4.2 Intelligence Artificielle.... 11
4.2.1 Observation.... 12
4.2.2 Actions....12
4.2.3 La fonction de récompense.... 13
4.2.4 Les Hyperparamètres.... 13
4.2.5 Inference....13
4.3 Rendu et Visualisation....14
4.3.1 Architecture du Rendu.... 14
4.3.2 Visualisation des Capteurs (Debug Visuel)....15
4.3.3 Mode "Ghost Car" et Transparence....15
5. Fine tuning du réseau de neurones....16
5.1 L'arrêt avant les virages....16
5.2 Manque d'expérience.... 16
5.3 Les virages en épingle....16
5.4 Retrait des Training Wheels.... 17
6. Conclusion....19
Annexes....19

1. Introduction

Dans le cadre de la SAE S5.A.01 de BUT informatique, il est demandé de réaliser un projet permettant de mettre en avant les compétences suivantes : “Réaliser un développement d’application” ; “Optimiser des applications” ; “Collaborer au sein d’une équipe informatique”.
Ce projet est encadré par mon tuteur technique M. DESBENOIT Brett et doit être réalisé sur un volume horaire de 195 heures.

Le sujet de projet choisi a été d’expérimenter avec l’apprentissage par renforcement au travers d’une simulation de voiture. L’ objectif est de créer une simulation physique cohérente du comportement d’une Formule 1, afin de servir de support pour la mise en œuvre d’un système de pilotage automatique capable de conduire même sur des circuits sur lesquels il ne s’est pas entraîné.

Ce projet vise à aborder des problématiques liées à l’expérimentation de méthodes issues du domaine de l’apprentissage automatique (machine learning), la conception d’une architecture adaptée et la création d’une simulation cohérente. Il s’inscrit dans une démarche pédagogique visant à mobiliser les compétences attendues du semestre 5 tout en permettant un approfondissement des connaissances en apprentissage automatique.

Ce rapport expose le développement complet d'un écosystème de conduite autonome, allant de la conception d'un moteur physique performant en C++ à l'implémentation d'un moteur d'inférence natif. Je détaillerai comment une architecture hybride a été mise en place pour permettre à une IA, entraînée via l'apprentissage par renforcement sous Python, de piloter une Formule 1 au sein d'une simulation. Le travail présenté couvre ainsi tout le cycle de vie du projet : la modélisation des forces physiques, la gestion des flux de données entre les modules et l’inférence de jeu en temps réel.

Ainsi, comment concevoir une architecture logicielle adaptée permettant de transformer une simulation physique en un environnement d'apprentissage efficace pour un agent autonome ?

2. Étude Préliminaire et Choix Technologiques

Avant d'entamer le développement, une phase de veille technologique a été nécessaire. Cette partie a impliqué la recherche et la définition des contraintes que j'allais rencontrer pour le projet. Cette étape m'a permis d'appréhender les subtilités du sujet et d'avoir une vision globale sur l'ensemble des goulots d'étranglement possibles que j'allais rencontrer. Cette veille a été réalisée en consultant la documentation officielle des outils que j'ai utilisés et en me servant de projets existants comme référence. J'ai également prototypé pour m'assurer que les outils utilisés correspondent à mes besoins.

Les choix suivants ont été faits en gardant à l'esprit que pour pouvoir entraîner un système d'apprentissage automatique, il est primordial de pouvoir simuler des millions de pas en un minimum de temps possible.

2.1 Le moteur physique

Le moteur physique est essentiel dans tout jeu simulant le comportement d'objets dans un monde et leurs intéractions entre eux. En tenant compte de lois physiques, il réalise des calculs permettant de déterminer les mouvements, les collisions et les réactions d'objets.

Pour répondre aux exigences de performances imposées par l'entraînement d'une IA sur cette simulation, j'ai choisi C++ comme langage pour le moteur physique car il offre les meilleures performances d'exécution parmi les langages dont je suis familier. Aussi, pour limiter la complexité de calculs, j'ai choisi de faire une simulation en deux dimensions. Le rendu 3d sera ensuite fait en mappant les positions et orientations des objets 2d.

Coder soi-même un moteur physique même 2d en tenant en compte la détection continue, la résolution de contraintes et l'optimisation de toutes les intéractions dans un monde simulé est un projet à part entière. Ainsi j'ai choisi d'utiliser la librairie Box2d. Box2d permet un contrôle précis sur les forces physiques au sein de ma simulation. Aussi, elle dispose d'un écosystème mature avec de nombreuses ressources et est optimisée pour des performances élevées. Ces raisons font que Box2d est une référence dans la simulation d'environnement.

2.2 Le module d'intelligence Artificielle.

L'écosystème de machine learning est aujourd'hui dominé par python. Tenter de réimplémenter un réseau de neurones en C++ avec de la rétropropagation aurait été très coûteux en temps de développement et une source d'erreur potentielle.

J’ai choisi d’utiliser l’apprentissage par renforcement pour entraîner mon IA. Ce type d’apprentissage permet à un agent d’apprendre à agir dans un environnement en ne recevant uniquement des récompenses ou pénalités, sans avoir besoin de données annotées. Cette approche est adaptée à un jeu de voiture, où l’agent peut expérimenter différentes trajectoires et améliorer progressivement ses performances.

La librairie stable baseline 3 basée sur PyTorch est un standard dans l’industrie pour sa fiabilité. Elle donne accès à de nombreux algorithmes efficaces pour l’apprentissage par renforcement ce qui m’a permis de me concentrer sur l'architecture du réseau et la définition des récompenses.

Parmi les algorithmes mis à disposition, deux candidats principaux se sont démarqués : DQN (Deep Q-Network) et PPO (Proximal Policy Optimization). Ces deux algorithmes permettent à un agent de prendre des décisions à partir d’input donné en maximisant les récompenses. La différence entre les deux vient du fait que DQN est bon pour les actions discrètes (accélérer, freiner, gauche, droite) et PPO pour les actions continues. J’ai choisi PPO pour permettre à la voiture d’avoir un contrôle plus fin sur ses actions. Au lieu de pouvoir seulement accélérer à 100% et tourner abruptement, à chaque step l'IA choisit un rayon de braquage entre -1.0 et 1.0.

2.3 Le pont entre l'IA et le moteur physique

L’utilisation de deux langages différents pour le module de training et de physique a impliqué de devoir faire un pont entre les deux pour avoir une communication efficace.

D’après mes recherches, j’ai trouvé deux façons de faire ce pont. La première consistait à faire communiquer le moteur physique et le module d’intelligence artificielle via un protocole inter-processus. Cela signifie que les processus d’intelligence artificielle et physique tournent en parallèle et communiquent au moyen de sockets sur localhost (protocole UDP). Pour une telle communication, il faut sérialiser les données en un format transmissible dans les deux sens. Ces aller-retours auraient été fait à chaque step de la simulation ce qui implique des latences considérables ce qui fait que cette solution n’est pas idéale.

Une autre solution est d’utiliser pybind11. Avec cette librairie, le moteur C++ est chargé comme un module Python. Cela signifie que les appels sont des appels de fonctions directs, les données restent en mémoire partagée réduisant significativement le coût par step. Cette méthode est supposée plus difficile à implémenter mais dans un contexte d’apprentissage par renforcement nécessitant des millions d’itérations, j’ai jugé que l’utilisation de pybind11 est justifiée.

2.4 Environnement de développement et automatisation

Pour la compilation du code en C++, j’ai utilisé CMake. Ce système de build m’a permis d’automatiser la configuration du projet, la gestion des bibliothèques externes et surtout un contrôle fin sur les flags de compilation ce qui m’a été très utile par la suite(En passant du flag DEBUG à RELEASE j’ai gagné 10x en fps sur la simulation).

Pour la gestion des dépendances en Python, j'ai choisi UV. UV est un gestionnaire de paquets et d'environnements réputé pour sa rapidité écrit en Rust. Son utilisation a permis de figer les versions des bibliothèques comme Stable Baseline 3 ou PyTorch via un fichier de verrouillage, garantissant ainsi que l'environnement d'entraînement reste identique sur différentes machines.

3. Architecture

3.1 Arborescence

```
.
├── assets
│   ├── shaders              # Fichiers pour le rendu graphique
│   └── tracks               # Données des circuits (fichiers CSV)
├── build
│   └── my_car_env.so        # Le module compilé (C++) importé dans Python
├── libs                     # Bibliothèques externes
│   ├── box2d                # Moteur physique
│   ├── imgui                # Interface graphique de debug
│   └── pybind11             # Pont entre C++ et Python
├── logs                     # Dossier de sortie pour les métriques
training
├── src                      # Cœur de l'application
│   ├── ai
│   │   ├── bindings.cpp     # Définition des points d'entrée pour Python
│   │   └── train.py         # Script principal d'entraînement (RL)
│   ├── engine
│   │   └── simulation.cpp   # Orchestration de la boucle de jeu
│   ├── physic
│   │   ├── car_physics.cpp  # Logique véhicule & Box2D
│   │   └── raycaster.cpp    # Capteurs de vision de l'IA
│   ├── render               # Gestion de l'affichage (OpenGL)
│   └── utils                # Parsers et outils (chargement des tracks)
├── CMakeLists.txt           # Configuration de la compilation C++
├── pyproject.toml           # Configuration du projet Python
└── uv.lock                  # Verrouillage des versions Python
```

Figure 1 : Arborescence du projet

L'arborescence du projet sépare clairement les codes sources (src/), les bibliothèques tierces (libs/), les assets de jeu (assets/) et les scripts de gestion de projet. Cette séparation des responsabilités facilite la maintenance et le travail itératif.

Le cœur de l'application se situe dans le dossier src/, structuré en quatre modules principaux assurant un découplage fort entre la logique et l'affichage :

- **src/physic/** : Contient la logique bas niveau. La classe CarPhysics encapsule les interactions avec Box2D, tandis que Raycaster gère les capteurs de la voiture (LIDAR). C'est ici que les calculs vectoriels et la détection de collision sont effectués.

- **src/engine/** : Orchestre la simulation via la classe Simulation. Elle fait le lien entre la physique de la voiture et l'environnement (le circuit).
- **src/render/** : Gère l'affichage via OpenGL/ImGui. Ce module est optionnel lors de l'entraînement de l'IA (mode "headless") pour économiser des ressources, illustrant une démarche d'optimisation.
- **src/ai/** : Contient le code spécifique au Machine Learning. Le fichier bindings.cpp est crucial : il définit l'interface Pybind11 qui expose les méthodes C++ (reset, step, get_observation) à Python.

Voir l'Annexe 1 : diagramme UML pour plus de détails.

3.2 Flux de Données et Pipeline d'Entraînement

L'interaction entre l'agent (IA) et l'environnement (Simulation) suit le standard Gymnasium utilisé par la bibliothèque Stable Baselines 3. L'architecture d'exécution se déroule comme suit :

A. **Initialisation :** Le script Python (src/ai/train.py) charge le module compilé (my_car_env.so situé dans build/). Il instancie l'environnement C++ et charge un circuit (par exemple assets/tracks/Monza.csv) via le TrackLoader (src/utils/).
B. **Boucle de Simulation (Step) :**
    - L'IA envoie une **action** (accélération, braquage) depuis Python.
    - bindings.cpp transmet cette action au moteur physique C++.
    - Le moteur physique simule un pas de temps (1/60ème de seconde).
    - Le module Raycaster lance des rayons pour mesurer les distances aux murs.
C. **Retour d'Information :**
    - L'état (distances des murs, vitesse) et la récompense (score) sont renvoyés à Python sans copie mémoire coûteuse grâce aux pointeurs partagés.
D. **Logging :** Les métriques de performance sont enregistrées dans le dossier logs/ par TensorBoard, qui permet d'avoir un tableau de bord très complet.

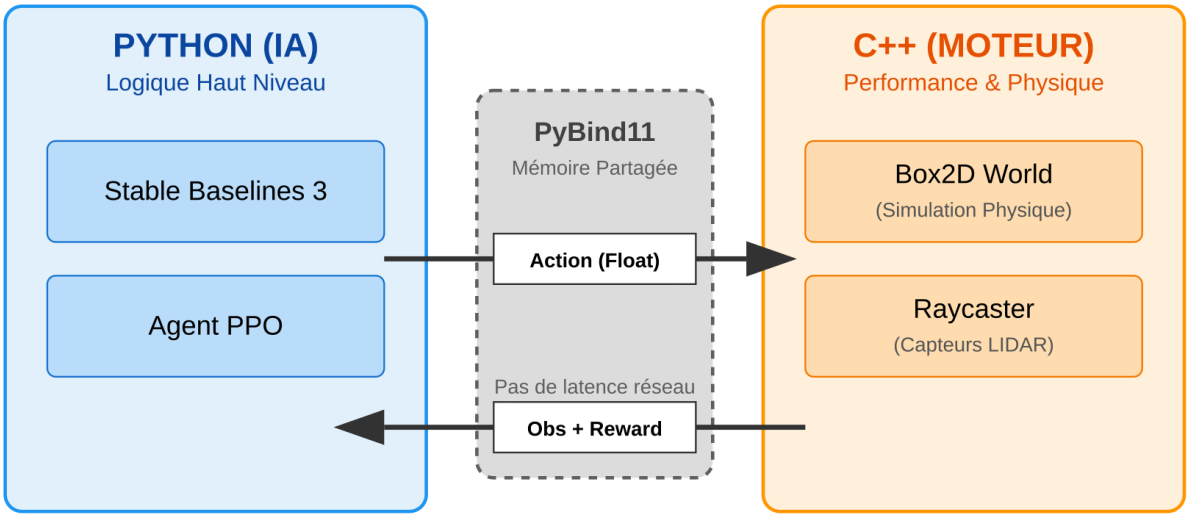


Figure 2 : flux de données entre l'IA et le moteur physique

4. Implémentation

4.1 Physique

Pour la Partie physique, j'ai essayé au maximum de simuler les données de vraies formule 1. Cela inclut la taille de la voiture qui fait 1.9m x 3.4m, la vitesse maximale de 350 km/h et différents phénomènes physiques simplifiés pour avoir un équilibre entre une simulation performante et réaliste.

4.1.1 Collisions

A 350 km/h, un objet peut facilement passer au travers de murs fins. Pour éviter ce phénomène, j'ai utilisé la détection continue de collision. Voici comment elle fonctionne sur la figure suivante:

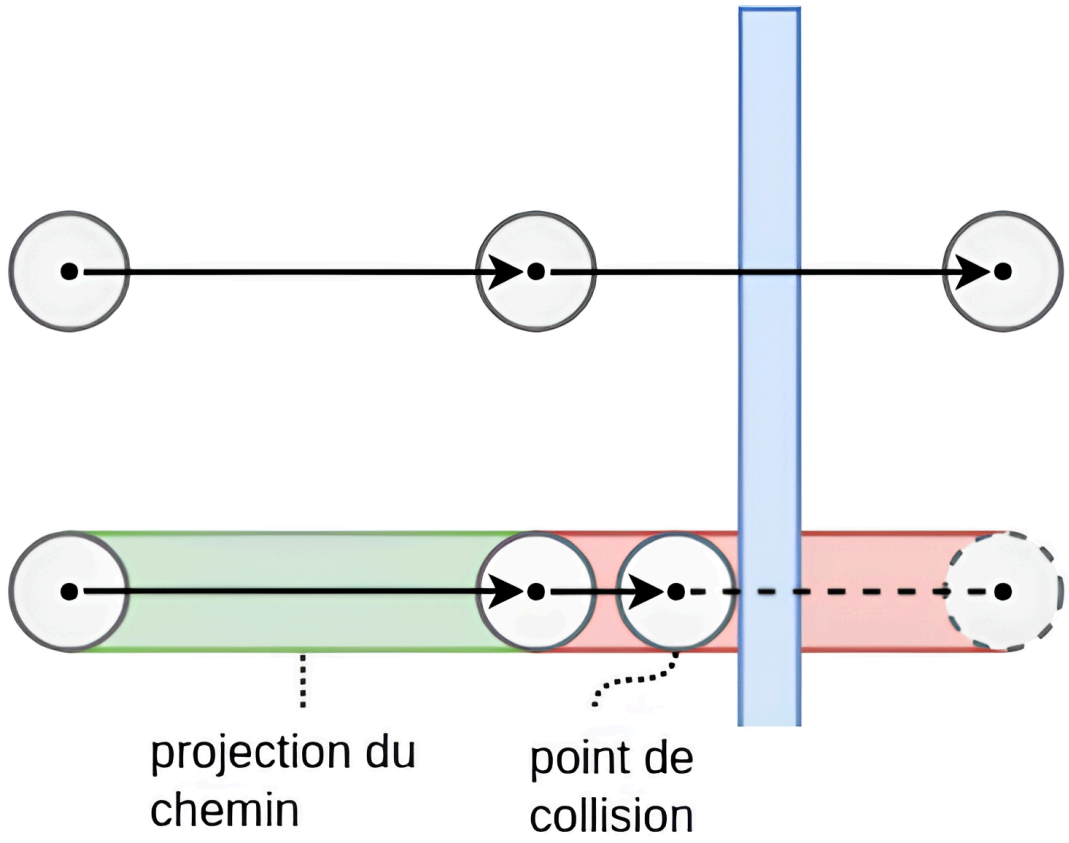


Figure 3 : Schéma de principe de la Détection Continue de Collision

4.1.2 Adhérence Latérale

Par défaut, dans une simulation Box2D, l'accélération s'applique directement au momentum dans la direction courante. Cependant, dans la vraie vie les pneus opposent une force contraire au glissement latéral.

Dans ma simulation, la méthode killOrthogonalVelocity applique une impulsion opposée afin d'annuler la vitesse latérale. Un coefficient est toutefois utilisé pour conserver un léger effet de drift.

```
void CarPhysics::killOrthogonalVelocity() {
    // Calcul de la vitesse latérale
    b2Vec2 sidewaysAxis = body->GetWorldVector(b2Vec2(1, 0));
    float sidewaysVelocity = b2Dot(velocity, sidewaysAxis);

    // Application d'une impulsion opposée
    // driftFactor = 0.98 permet de garder un léger dérapage
    b2Vec2 impulse = -sidewaysVelocity * driftFactor * body->GetMass() * sidewaysAxis;
    body->ApplyLinearImpulse(impulse, body->GetWorldPoint(localPoint), true);
}
```

4.1.3 Simulation de la direction

Plutôt que de tourner via une force brute, je calcule un rayon de braquage dynamique selon la vitesse : plus la voiture va vite, plus elle a besoin d'espace pour virer. Cela simule le sous-virage : à pleine vitesse, elle ne peut pas suivre l'angle demandé et doit freiner avant le virage.

```
void CarPhysics::update(CarInput input) {
    // ... (Code d'accélération linéaire) ...

    // Calcul du rayon de braquage dynamique pour simuler le sous-virage
    // Rayon = Base + (Vitesse * Facteur)
    float dynamicRadius = baseTurnRadius + (absSpeed * speedTireFactor);

    // Calcul de la vitesse angulaire physique (Omega = V / R)
    float desiredAngularVelocity = (currentForwardSpeed / dynamicRadius) *
-input.steering;

    // Application de l'impulsion de rotation
    float impulse = body->GetInertia() * (desiredAngularVelocity -
body->GetAngularVelocity());
    body->ApplyAngularImpulse(impulse, true);
}
```

4.2 Intelligence Artificielle

L'entraînement du modèle s'est fait avec l'algorithme PPO implémenté avec la bibliothèque Stable Baseline 3.

L’exécution de l'algorithme s’est faite uniquement avec le cpu. En effet, pour un petit réseau de neurones, le gain en temps de calcul obtenu en utilisant le gpu ne contrebalance pas le coût de transfert des données.

4.2.1 Observation

Pour que l'IA puisse naviguer, elle doit percevoir son environnement. Pour se faire, à chaque step elle reçoit la vitesse de la voiture, son angle et sa distance aux murs.

Pour la distance aux murs, j’ai implémenté un système de Raycasting en C++ qui simule un capteur LIDAR.

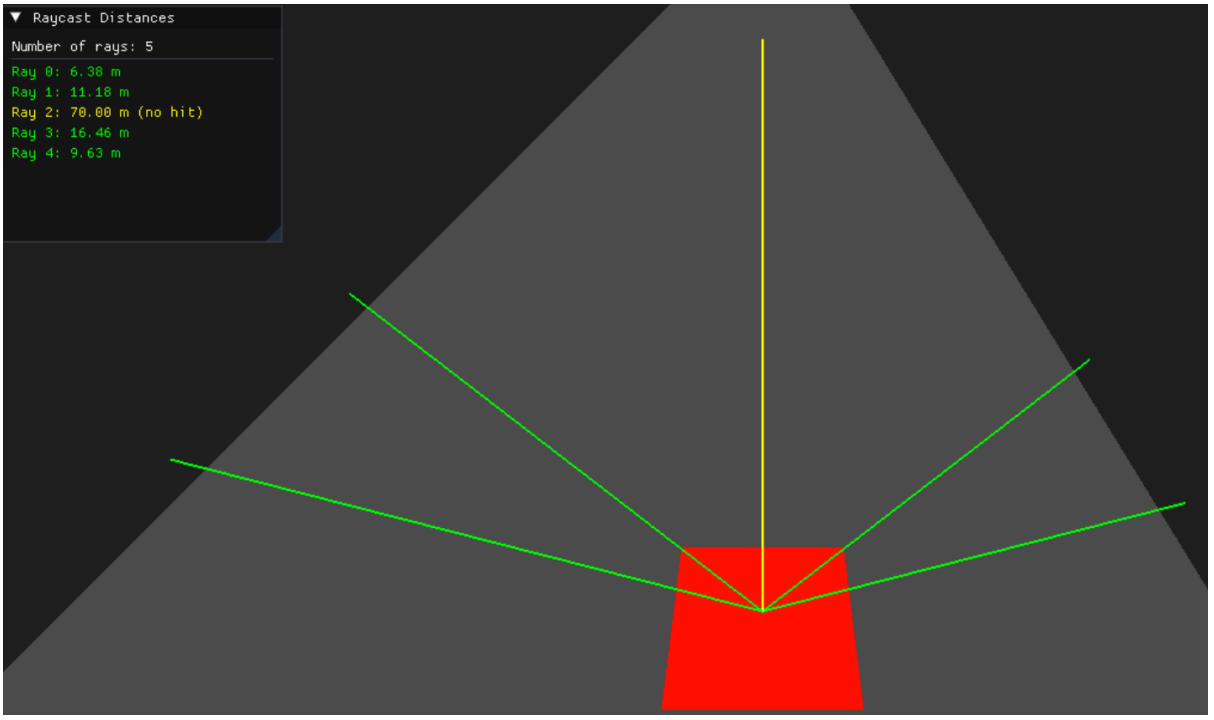


Figure 4 : Visualisation des 5 rayons du LIDAR

Les données brutes (en mètres) sont systématiquement divisées par la portée maximale des capteurs afin de fournir au réseau des valeurs normalisées entre 0 et 1.

4.2.2 Actions

À chaque step de la simulation, l'IA retourne deux valeurs flottantes : Steering : [-1.0, 1.0] (Gauche <-> Droite) et Throttle : [-1.0, 1.0] (Freinage <-> Accélération). Cela lui permet une conduite fluide et précise.

Il est important d’avoir des valeurs normalisées car cela facilite grandement la convergence du réseau de neurones. Les algorithmes d'optimisation comme la descente de gradient sont instables lorsque les ordres de grandeur varient trop. En contraignant

les sorties entre -1 et 1, on assure que le poids de l'accélération n'écrase pas celui de la direction dans les calculs d'erreur.

4.2.3 La fonction de récompense.

La fonction de récompense c’est ce qui permet de définir les objectifs de l’agent. L’algorithme de PPO cherche à maximiser la récompense. Il collecte les trajectoires, calcule l’avantage de chaque action prise par rapport à la moyenne, puis ajuste les poids du réseau de neurones pour augmenter la probabilité de reproduire les actions ayant généré les meilleures récompenses.

Si la fonction de récompense est mal définie, on a un comportement qui maximise la récompense oui mais qui ne correspond pas à l’objectif attendu. Par exemple, si la pénalité de collision et la récompense de survie sont trop élevées, l'IA peut décider à chaque virage de s’arrêter car essayer de passer le virage n’est pas rentable.

4.2.4 Les Hyperparamètres

Les hyperparamètres sont les variables de configuration qui régissent le comportement de l'algorithme PPO. C’est la façon dont apprend l’IA.
Ils influencent la stabilité et la convergence du modèle. Si mal configurés on peut notamment rencontrer des régressions dans la qualité du modèle.

Ils influencent notamment la vitesse à laquelle le réseau modifie ses connexions, la vision à long terme (perdre en récompense en freinant pour en gagner plus en réussissant un virage) ou encore sa tendance à essayer de nouvelles stratégies.

Voir en annexe 2 la configuration complète des hyperparamètres de mon modèle.

4.2.5 Inference

Afin de voir les résultats de l'entraînement (autrement que sur des graphiques de récompense), j’ai implémenté un moteur d’inférence. Le principe est d'exporter les poids du réseau de neurones entraîné vers un fichier binaire, puis de les recharger dans le programme C++ principal.

Cette implémentation permet de reconstruire l'architecture illustrée ci-dessus, où les données des capteurs transitent par trois couches cachées décroissantes (512, 256, 128 neurones) avant de déterminer l'accélération et la direction.

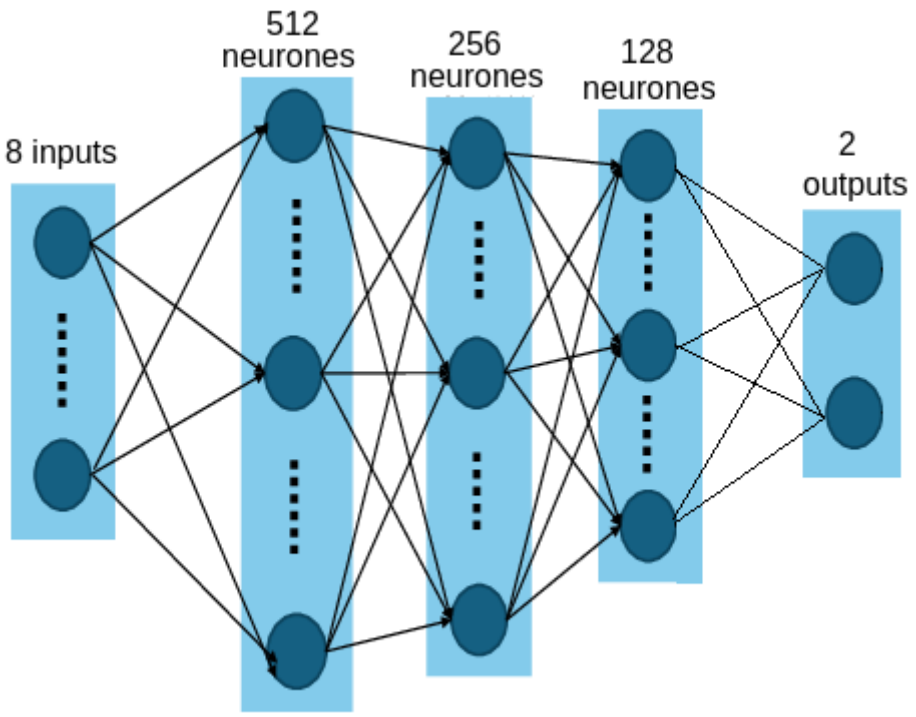


Figure 5 : Architecture du Perceptron Multicouche (MLP) utilisé pour l'inférence

4.3 Rendu et Visualisation

Bien que l'entraînement de l'IA se fasse en headless (sans rendu), il a tout de même été nécessaire d'implémenter une classe qui gère le rendu pour pouvoir déboguer la physique et avoir un visuel sur ce que fait l'IA.
Pour ce faire j'ai implémenté la classe SimpleRenderer qui utilise l'API OpenGL pour le rendu.

4.3.1 Architecture du Rendu

Le rendu est découplé de la logique physique. La classe SimpleRenderer gère la compilation des shaders (Vertex et Fragment) et la gestion des tampons mémoire (VBO/VAO).

- Shaders : J'utilise des shaders GLSL qui gèrent la projection 3D et la couleur. Un système d'uniformes permet de modifier dynamiquement la transparence (u_Alpha), crucial pour la visualisation "Ghost".
- Batching : Le circuit est généré une seule fois sous forme de mesh statique (GL_STATIC_DRAW), tandis que les éléments mobiles comme la voiture ou les rayons sont mis à jour à chaque frame.

4.3.2 Visualisation des Capteurs (Debug Visuel)

Pour comprendre pourquoi l'IA prend certaines décisions, il est impératif de voir ce qu'elle voit. J'ai implémenté un système de visualisation des rayons du LIDAR utilisant GL_LINES avec des buffers dynamiques (GL_DYNAMIC_DRAW).

Le rendu s'adapte à l'état de la perception :

Rayon Vert : Le capteur détecte un mur à portée.

Rayon Jaune : Le capteur ne détecte rien (portée maximale atteinte).

Cela permet de vérifier instantanément si l'IA échoue un virage à cause d'une mauvaise décision ou d'une mauvaise perception.

4.3.3 Mode "Ghost Car" et Transparence

Afin de comparer différentes itérations de l'IA ou de visualiser l'inférence par rapport à une trajectoire idéale, j'ai implémenté une fonctionnalité de "Voiture Fantôme" (drawGhostCar). En activant le blending OpenGL (glEnable(GL_BLEND)), j'ai superposé une seconde voiture semi-transparente.

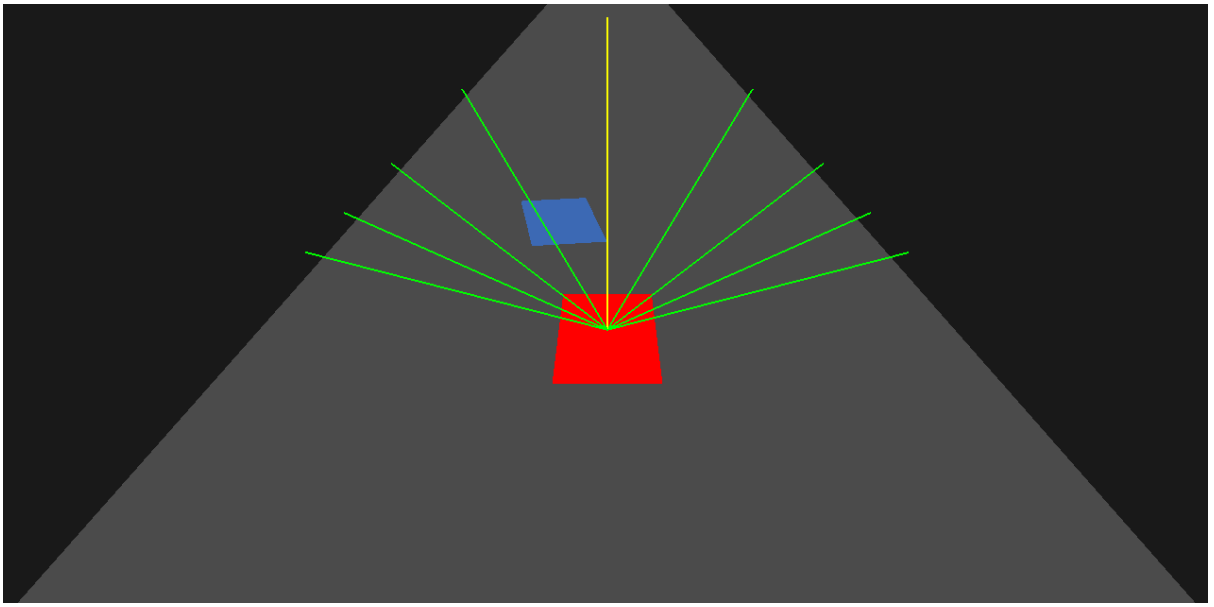


Figure 6 : Rendu de la voiture fantôme

5. Fine tuning du réseau de neurones

Une fois tous les éléments précédents en place, vient la phase d'expérimentation pour améliorer l'IA. Cette phase demande l'ajustement des récompenses, des données d'entrée et de l'architecture du réseau à chaque itération.

5.1 L'arrêt avant les virages

Les premières sessions d'entraînement ont révélé des failles dans la définition de la fonction de récompense, exploitées par l'IA. Pour ne pas recevoir de pénalité, l'IA a appris à s'arrêter dès qu'elle approche un virage.

Pour remédier à ce problème, j'ai implémenté une pénalité lorsque la voiture est à l'arrêt. Ce qui a effectivement permis à la voiture de toujours être en mouvement.

5.2 Manque d'expérience

Maintenant, bien que la voiture essaie de passer le premier virage, elle n'y parvenait pas. Par conséquent, elle ne prenait aucune nouvelle stratégie car elle ne connaît qu'une partie d'un circuit, elle ne rencontre ni de courbe ni rien.

Pour briser ce cycle, j'ai diversifié les environnements d'entraînement en ajoutant 7 pistes différentes. A ce stade, l'IA parvenait à finir les circuits les plus simples.

5.3 Les virages en épingle

À ce stade, l'IA avait une conduite performante cependant elle échouait systématiquement les virages en épingles. Pour résoudre ce problème j'ai essayé diverses solutions :

Hypothèse 1 : L'IA ne perçoit pas suffisamment son environnement. J'ai donc augmenté le nombre de rayons de raycasting.
Résultat : Aucun changement significatif sur les virages en épingle.

Hypothèse 2 : Manque d'exploration. J'ai augmenté le nombre d'époques (n_epochs) et le coefficient d'entropie pour encourager l'IA à tester de nouvelles trajectoires.
Résultat : L'entraînement est devenu instable, il y avait parfois des régressions dans les récompenses.

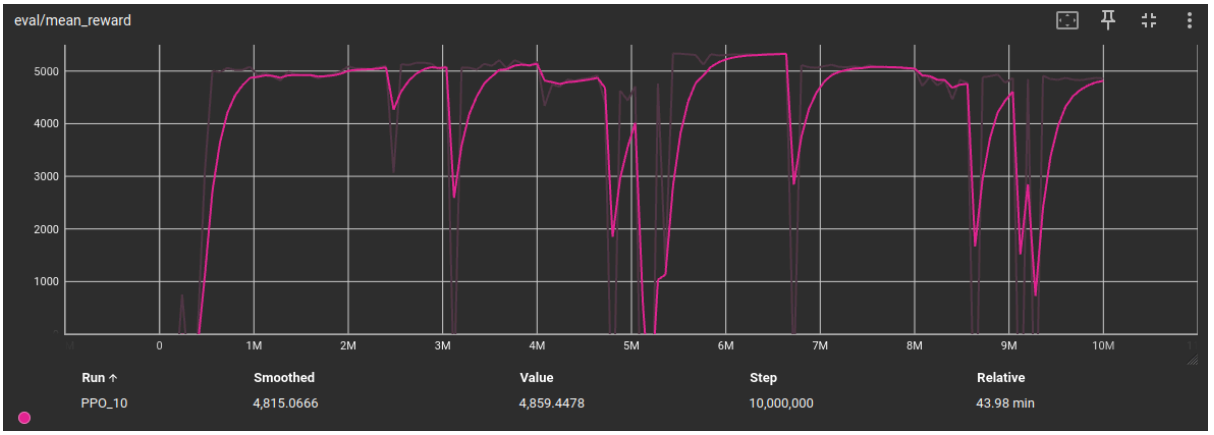


Figure 7 : Évolution de la récompense moyenne lors de l'expérimentation de l'Hypothèse 2

L'instabilité marquée de la courbe, caractérisée par des effondrements fréquents et brutaux du score, démontre que l'augmentation excessive du coefficient d'entropie et du nombre d'époques nuit à la convergence du modèle et empêche le maintien des performances acquises.

Hypothèse 3: Le réseau est trop simpliste et ne parvient pas à gérer des situations complexes. Je suis donc passé d'un réseau avec deux couches cachées de 256 neurones à un réseau avec trois couches de 512, 256, 128.
La première couche plus large permet d'ingérer une grande quantité de nuances dans les signaux bruts des capteurs.
De plus, l'architecture en entonnoir force le réseau à compresser l'information et à abstraire les données. Au lieu de réagir bêtement à un rayon, l'IA apprend à reconnaître des "concepts" de haut niveau (comme la géométrie d'une épingle) en éliminant le bruit inutile.
Résultat : L'IA parvient désormais à prendre tous les virages et ne se crash plus.

5.4 Retrait des Training Wheels

Jusqu'à présent, mon modèle recevait une faible récompense lorsqu'il restait au milieu de la piste. Par conséquent, la voiture ne prenait jamais la corde pour gagner du temps.

J’ai donc retiré cette punition et voici la comparaison des récompenses moyennes avec et sans ces “training wheels”

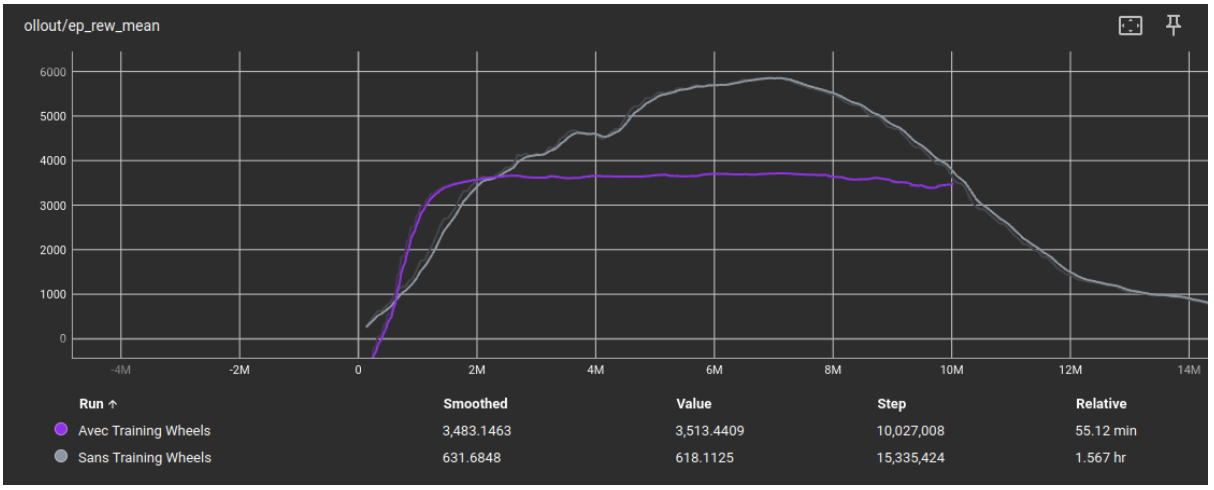


Figure 8 : Comparaison des courbes d'apprentissage : stabilisation avec aide au centrage (violet) vs effondrement sans aide (gris)

Avec l'incentive pour rester au milieu, l'IA progresse et atteint un plateau. Pour gagner plus de points, elle devrait couper les virages . Mais si elle le fait, elle perd son bonus de centrage. Elle est donc "coincée" dans une conduite sûre mais sous-optimale. Elle ne progresse plus, mais elle ne régresse pas.

Sans l’incentive, effectivement l'IA va plus vite et maximise les récompenses comme théorisé mais on peut voir qu’après le sommet à 7 millions de step, le modèle régresse grandement. Ce phénomène est ce qu’on appelle ‘policy collapse’ en reinforcement learning. Cela peut être dû à un sur-apprentissage sur des données récentes, mises à jour de politique trop agressives ou encore une instabilité dans l'optimisation PPO.

6. Conclusion

Ce projet, réalisé dans le cadre de la SAE S5.A.01, m'a permis d'explorer comment combiner simulation physique et apprentissage par renforcement. Mon objectif était de créer une IA capable de piloter une voiture de course de manière autonome, même sur des circuits qu'elle n'a jamais vus. Et finalement, j'y suis arrivé, ce qui prouve que l'architecture que j'ai mise en place fonctionne bien.

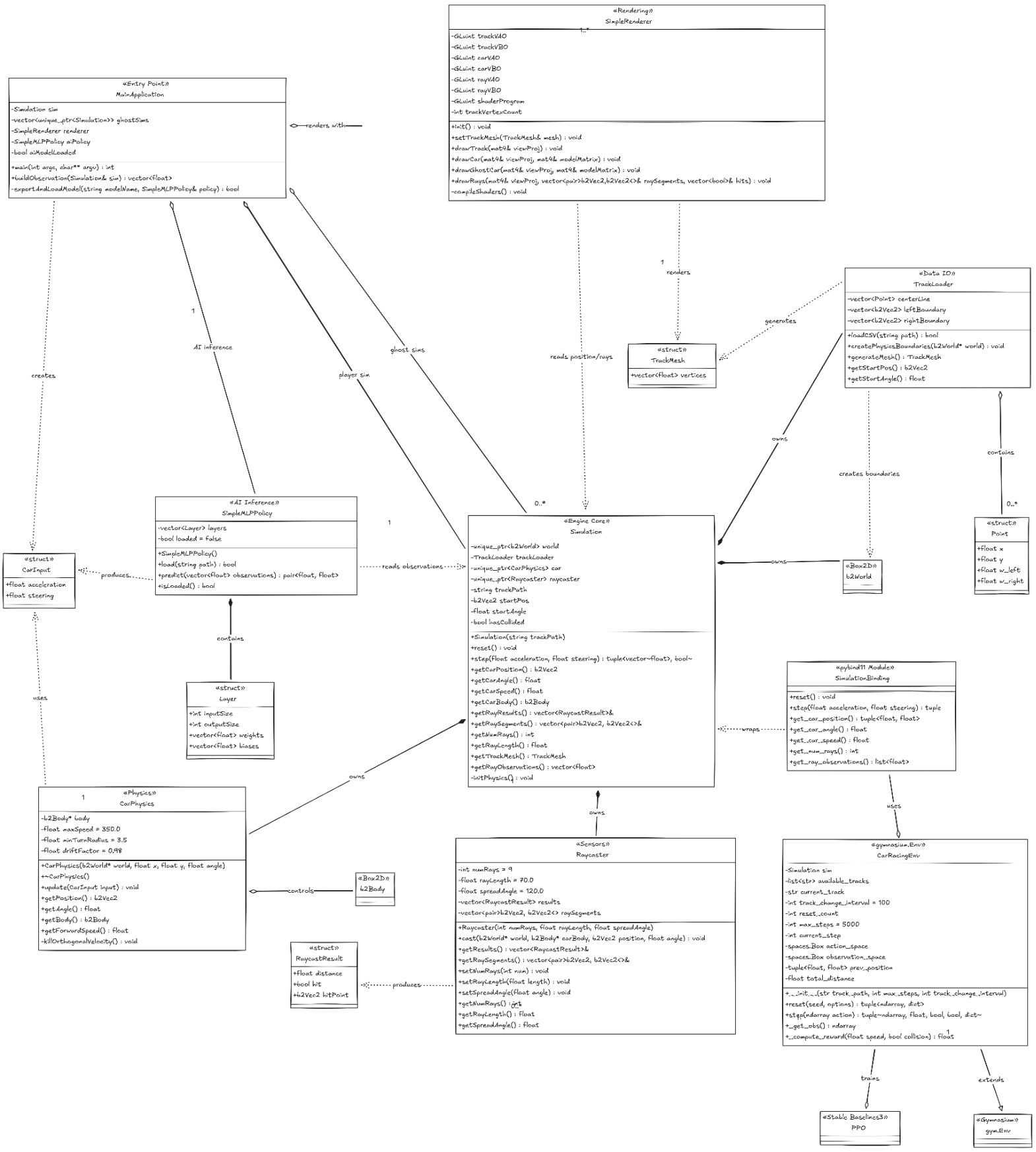
D'un point de vue technique, j'ai vraiment pu travailler sur la conception d'applications complexes. Le plus gros challenge a été de faire le pont entre deux systèmes complexes: le C++ pour avoir une simulation physique rapide avec Box2D, et Python pour entraîner l'IA avec Stable Baselines 3. Utiliser Pybind11 pour créer ce pont entre les deux s'est avéré être un excellent choix, ça m'a permis d'avoir un entraînement fluide qui tourne à plusieurs milliers d'images par seconde.

L'optimisation a été le fil conducteur de mon projet, de la simulation jusqu'à l'exécution finale. J'ai d'abord maximisé les performances du moteur C++ en affinant les flags de compilation et en simplifiant les lois physiques pour gagner en fluidité. Grâce à l'analyse des courbes TensorBoard à chaque itérations, j'ai pu ajuster avec précision la fonction de récompense et les capteurs d'entrée. Enfin, j'ai développé un moteur d'inférence natif en C++ pur : en réimplémentant mathématiquement le réseau de neurones pour l’inférence.

Ce projet m'a aussi beaucoup appris sur l'organisation et l'auto-formation. Face à tous ces concepts complexes, que ce soit les maths derrière les réseaux de neurones ou la physique pour simuler une voiture réaliste, j'ai dû structurer ma veille technologique et avancer par essais-erreurs. Chaque blocage m'a forcé à chercher, comprendre et itérer jusqu'à trouver la solution.

Sur cette base pour le prochain semestre, mon intention est d’améliorer le rendu en ajoutant des modèles plus complexes et d’améliorer la mécanique de jeux incluant un système de chronométrage par tours, ainsi qu’une interface permettant la sélection de différentes pistes. Je pourrais également explorer l’utilisation d’algorithmes différents dans l'entraînement de l’agent de conduite.

Annexes



Annexe 1: Diagramme UML

```python
model = PPO(
    "MlpPolicy",
    env,
    device="cpu",
    verbose=1,
    learning_rate=1e-4,   # Taux faible pour un apprentissage stable
    n_steps=4096,         # Grand buffer pour de meilleures estimations de gradient
    batch_size=128,       # Taille de lot augmentée pour la stabilité
    n_epochs=15,          # Nombre de passes d'optimisation par mise à jour
    gamma=0.995,          # Horizon long terme (essentiel pour anticiper les virages)
    gae_lambda=0.98,      # Lissage de l'estimation de l'avantage
    clip_range=0.15,      # Clipping serré pour des mises à jour prudentes
    ent_coef=0.08,        # Entropie élevée pour forcer l'exploration
    vf_coef=0.5,          # Importance de la fonction de valeur
    max_grad_norm=0.5,    # Limite la norme du gradient (anti-explosion)
    policy_kwargs=dict(
        # Architecture du réseau de neurones : 3 couches cachées
        # Pi : Réseau de politique (Action)
        # Vf : Réseau de valeur (Estimation du gain)
        net_arch=dict(pi=[512, 256, 128], vf=[512, 256, 128])
    )
)
```

Annexe 2 : Configuration du Modèle PPO